

Requirements Life Cycle

1. Requirement Definition for an E-Commerce Website

Context: A company wants to develop an e-commerce website to sell its products online.

Scenario of Requirement Definition:

1. Identifying Stakeholders:

- The project team identifies key stakeholders, including the **business owner**, **customers**, **marketing team**, and **IT staff**.

2. Gathering Requirements:

- The team conducts interviews and surveys with stakeholders to understand their needs and expectations. They ask questions like:
 - What types of products will be sold?
 - What payment methods should be supported?
 - Are there specific shipping options required?

3. Understanding the Application Domain:

- The team researches the **e-commerce industry** to learn about common features and best practices, such as product reviews, wish lists, and customer accounts.

4. Defining Functional Requirements:

- Based on discussions, the team defines functional requirements, including:
 - Users should be able to create accounts and log in.
 - Users should be able to search for products and filter results.
 - Users should be able to add items to a shopping cart and proceed to checkout.

5. Identifying Non-Functional Requirements:

- The team also identifies non-functional requirements, such as:
 - The website should load within 3 seconds.
 - The system should be secure and protect user data.

6. Understanding Operational Constraints:

- The team learns about constraints, like:
 - The website must be compatible with mobile devices.
 - It must comply with local laws regarding online sales.

7. Documenting Requirements:

- All gathered requirements are documented clearly, creating a **Requirement Specification Document** that serves as a reference for developers and designers.

2. Requirement Validation

Requirement Validation is a critical phase in which the gathered requirements are checked to ensure they are acceptable, feasible, verifiable, and aligned with business goals. The aim is to prevent costly errors and misunderstandings before the development process begins or while it is still early enough to fix issues.

Example:

1. **Scenario:** A food delivery platform is in the development phase, and a requirement states, "The system should allow users to place orders from multiple restaurants in a single transaction."
2. **Requirement Validation Process:**
 - **Feasibility Check:** The technical team evaluates whether this multi-restaurant ordering feature can be implemented within the platform's existing infrastructure. They assess whether the payment gateway supports this functionality and if any additional complexities arise (like different delivery times).
 - **Stakeholder Review:** The business analyst verifies this requirement with restaurant partners and internal business teams to ensure it aligns with their business model. Restaurant owners may prefer separate orders for each restaurant to avoid logistical issues.
 - **Testability Check:** The quality assurance team checks if this requirement can be tested in a real-world scenario and determines how they will verify that multiple orders are processed correctly in a single transaction.
3. **Outcome:** After validation, the team may realize that the requirement is not feasible within the current budget or timeline. Instead, the requirement may be adjusted to only allow orders from one restaurant at a time in the initial launch, with the possibility of expanding later.

Purpose of Validation:

- **Catch issues early:** Ensures problems like technical feasibility, conflicting business rules, or unverifiable requirements are caught early before resources are wasted.
- **Improve cost efficiency:** Prevents rework and major changes during or after development, reducing overall costs.
- **Ensure clarity:** Confirms that everyone—business, technical, and quality assurance teams—has a clear, shared understanding of what needs to be built.

3. Requirement Documentation

Scenario: Development of an E-Commerce Website

1. Functional Requirements:

- *User* *Registration:*
The system must allow users to register using their email, social media accounts, or mobile number.
 - **Priority:** High
 - **Reason:** Registration is required for users to make purchases and track orders.
- *Product* *Search:*
The system should allow users to search for products using keywords, categories, or filters (such as price range, brand, etc.).
 - **Priority:** High
 - **Reason:** This feature enhances the user experience by simplifying the search process.
- *Payment* *Gateway* *Integration:*
The system must support multiple payment methods (credit cards, PayPal, bank transfers).
 - **Priority:** Medium
 - **Reason:** Providing users with multiple payment options will increase customer satisfaction and conversion rates.

2. Non-functional Requirements:

- *Performance* *Speed:*
The website must load within 3 seconds for 95% of users under normal conditions.
 - **Priority:** High
 - **Reason:** Fast loading times are essential for reducing bounce rates and improving user retention.
- *Security:*
The system must ensure all payment and personal information is encrypted and comply with PCI-DSS standards.
 - **Priority:** High
 - **Reason:** Security is critical for maintaining trust with users and protecting sensitive information.
- *Scalability:*
The system should be scalable to support up to 50,000 concurrent users during peak times.
 - **Priority:** Medium
 - **Reason:** Scalability is important to accommodate traffic spikes during promotions or holidays.

3. System Constraints:

- The system must be built using a cloud-based architecture to ensure high availability and disaster recovery.
- The system must support integration with third-party logistics providers for real-time shipping updates.
- Development must be completed within 6 months, with a budget of \$500,000.

4. Assumptions:

- The majority of users will access the system through mobile devices, so mobile responsiveness is essential.
- Payment processing will be handled by external gateways, so the system does not need to store sensitive credit card information.

Outcome of the Requirement Documentation:

- Developers have a clear framework to follow during the implementation phase, knowing exactly what features to develop and what constraints to work within.
- Future changes to the system will be easier to manage since the requirements have been documented, providing a baseline for evaluating new feature requests or improvements.
- Any miscommunications between stakeholders and developers are minimized as all features, non-functional requirements, and constraints are clearly outlined.

4. Mobile Banking Application Development

Context: A company is developing a mobile banking app to improve user experience.

Scenario of Poor Requirement Management:

1. **Gathering Requirements:**
 - The team talks to users to collect their needs, but many important features, like **data security**, are missed because they weren't written down properly.
2. **Adding New Features:**
 - Halfway through development, stakeholders ask for a new feature, **mobile check deposit**, without considering its impact on the existing design. This causes confusion and delays.
3. **Lack of Traceability:**
 - Requirements aren't linked to specific features, making it hard to figure out why the **login function** has bugs. The security details were unclear.
4. **Final Product Issues:**
 - When the app launches, users face problems, such as security vulnerabilities and broken features. This causes them to lose trust in the app.

5. **User Feedback:**

- After the launch, many users complain about security issues, leading to a rushed effort to fix bugs, which makes the app even less reliable.